



Design and Realisation of Scalable Business Process Management Systems for Deployment in the Cloud

CHUN OUYANG, MICHAEL ADAMS, and ARTHUR H. M. TER HOFSTEDÉ, School of Information Systems, Queensland University of Technology, Australia
YANG YU, School of Data and Computer Science and School of Software, Sun Yat-sen University, China

36

Business Process Management Systems (BPMSs) provide automated support for the execution of business processes in modern organisations. With the emergence of cloud computing, BPMS deployment considerations are shifting from traditional on-premise models to the **Software-as-a-Service (SaaS)** paradigm, aiming at delivering Business Process Automation as a Service. However, scaling up a traditional BPMS to cope with simultaneous demand from multiple organisations in the cloud is challenging, since its underlying system architecture has been designed to serve a single organisation with a single process engine. Moreover, the complexity in addressing both the dynamic execution environment and the elasticity requirements of users impose further challenges to deploying a traditional BPMS in the cloud. A typical SaaS often deploys multiple instances of its core applications and distributes workload to these application instances via load balancing. But, for *stateful* and often long-running process instances, standard stateless load balancing strategies are inadequate. In this article, we propose a conceptual design of BPMS capable of addressing dynamically varying demands of end users in the cloud, and present a prototypical implementation using an open source traditional BPMS platform. Both the design and system realisation offer focused strategies on achieving scalability and demonstrates the system capabilities for supporting both upscaling, to address large volumes of user demand or workload, and downscaling, to release underutilised computing resources, in a cloud environment.

CCS Concepts: • **Information systems** → **Information systems applications**; • **Applied computing** → **Business process management systems**;

Additional Key Words and Phrases: Business process management systems, software-as-a-service, scalability, business process engine, load balancing

ACM Reference format:

Chun Ouyang, Michael Adams, Arthur H. M. ter Hofstede, and Yang Yu. 2021. Design and Realisation of Scalable Business Process Management Systems for Deployment in the Cloud. *ACM Trans. Manag. Inform. Syst.* 12, 4, Article 36 (September 2021), 26 pages.
<https://doi.org/10.1145/3460123>

This work is supported by the Research Foundation of Science and Technology Plan Project in Guangdong Province under Grant No. 2020A0505100030 and in part supported by the Australian Research Council Discovery Grant No. DP190100314. Authors' addresses: C. Ouyang, M. Adams, and A. H. M. ter Hofstede, School of Information Systems, Queensland University of Technology, 2 George Street, Brisbane, Queensland 4000, Australia; emails: {c.ouyang, mj.adams, a.terhofstede}@qut.edu.au; Y. Yu, School of Data and Computer Science and School of Software, Sun Yat-sen University, 132 Waihuan East Road, Guangzhou Higher Education Mega Center, Guangzhou 510006, China; email: yuy@mail.sysu.edu.cn. Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

2158-656X/2021/09-ART36 \$15.00

<https://doi.org/10.1145/3460123>

1 INTRODUCTION

Cloud computing, as a major computing paradigm emerged over the last decade, has evolved into one of the most widely used computing resource platforms, and has made a significant impact on how organisations run their business applications. The emphasis on resource sharing and rapid provision of high-level services with minimal management effort in the cloud is empowering organisations, in particular small and medium-sized enterprises, with the capability to deploy their business processes through cloud-based service provision instead of the traditional on-premise **Business Process Management Systems (BPMSs)**.

BPMSs, as a key exemplar of process-aware information systems, are dedicated to providing automated support for the execution of business processes in modern organisations, and deliver many proven benefits [22]. The additional advantages offered by cloud computing have triggered an increased demand for BPMS deployment to shift from traditional on-premise modes to the **Software-as-a-Service (SaaS)** paradigm, with an aim to deliver *Business Process Automation as a Service* to multiple organisations in a multi-tenant cloud.

However, the architecture of traditional on-premise BPMSs is designed to serve a single organisation and typically rely on a single process engine. In a multi-tenant cloud, scaling up a discrete BPMS to address simultaneous demands from multiple organisations is challenging. To cope with the increasing load due to multi-tenancy, a SaaS approach often deploys multiple instances of its core applications and distributes the workload of tenant requests to these application instances via load balancing [3]. The assumption in this approach is that the tenant requests are stateless, whereas the execution of a business process is *stateful and often long-running*, thus existing load balancing strategies for handling stateless requests are rendered inadequate. Moreover, the complexity in addressing both the dynamic execution environment and the elasticity requirements of users [5] impose further challenges to deploying a traditional BPMS in the cloud.

These challenges have motivated the need for the redesign of an architectural framework for BPMS to be ready for deployment in the cloud. The resultant system design would be expected to support *scalability* and *elasticity* (i.e., being capable of automatically scaling operations up and down to ensure optimal availability and utilisation of resources) and *stateful* process execution in a BPMS deployed in the cloud. Such a system design should also be *independent* of any specific BPMS or cloud computing platform. Supporting business process execution in a cloud environment (sometimes referred to as “BPM in the cloud” [18]) has emerged as a relatively new field of research in recent years, and a review of the existing efforts towards the design of a general, scalable system architecture for deploying BPMS in the cloud reveals that this aim has not yet been achieved.

In this article, we propose a conceptual design of BPMS for deployment in a cloud environment with a focus on supporting scalability and stateful process execution and not relying on any specific system and cloud platform. We adopt an established design science research framework for conducting our design activities. The proposed design consists of four conceptual specifications capturing the system architecture, data requirements, sequences of interactions between system components, and load distribution capability levels. To illustrate the transformation of our conceptual design specification into an effective system realisation, a prototype is developed leveraging an open source BPMS environment that was built upon a traditional architecture prior to the proposal in this article. Simulations of multiple process execution scenarios with largely varying workloads, which mimic the elasticity demands of users in a dynamic execution environment, demonstrate that the system prototype is capable of both upscaling to address large volumes of workload on demand and downscaling to release underutilised cloud computing resources whenever possible.

The rest of the article is organised as follows. Section 2 presents the adopted research methodology and framework. Section 3 describes the relevant background knowledge from which design

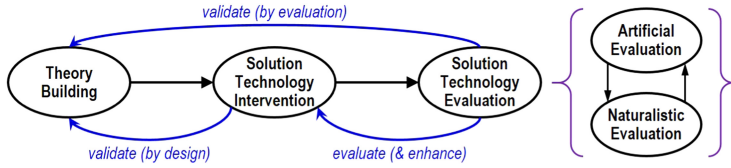


Fig. 1. Framework of design science research activities (adapted from [25]).

requirements are drawn. Section 4 proposes the conceptual design of BPMS for deployment in a cloud environment. Section 5 discusses the realisation of a system prototype and the implementation of different workload distribution strategies. Section 6 presents the design of experiments for evaluation and discusses the corresponding simulation results. Section 7 reviews related research efforts. Finally, Section 8 concludes the article and sets out future directions.

2 RESEARCH METHODOLOGY

We apply a design science research methodology [10] and adopt the framework for design science research activities proposed by Venable [25] to guide our research in this article. As depicted in Figure 1, the framework consists of *theory building*, *solution technology intervention*, and *solution technology evaluation* via two types of evaluation activities, namely, *artificial evaluation* and *naturalistic evaluation*. Starting from theory building, an adequate theory, including relevant background and domain knowledge, needs to be selected that provides an explanation about the chosen topic of interest. This results in a set of requirements on how the solution technology should be designed. Based on the requirements, the solution technology will be developed, and this is the core of the design science research. Next, the proposed solution technology should be evaluated. This can be conducted in a simulated setting (e.g., computer simulations, lab experiments), also known as artificial evaluation; or using real examples (e.g., case studies, field studies) also known as naturalistic evaluation. The results of evaluation can be used to understand the performance of the solution technology, and also fed back to the theory building activity to (dis-)confirm the fulfilment of the requirements (i.e., validation of the solution technology). As such, it may lead to drawing useful insights to enhance the proposed solution technology.

In our research, as guided by the above framework of research activities, we first study the relevant background knowledge regarding BPMS and cloud computing, since our focus is on designing BPMS that is ready for deployment in the cloud. This leads us to define a set of requirements the proposal of our design should address (Section 3). Based on the requirements, we develop our solution technology, which comprises a conceptual design of BPMS (Section 4) and a prototypical implementation transforming the conceptual design into a system realisation (Section 5). Next, we evaluate the proposed design by systematically carrying out a set of experiments using the implemented prototype with simulated scenarios. Findings drawn from the analysis of the evaluation results are used to validate the proposed solution against certain requirements, and may also inform possible refinement or extension to the proposed solution (Section 6).

3 BACKGROUND AND REQUIREMENTS

3.1 Principles of BPMS

A BPMS aims to support business process execution by coordinating the right work to the right person (or application) at the right time. Business process execution is about carrying out a set of tasks, following a specific order of occurrences, to achieve a business goal. A task is a logic unit of work performed by a resource (e.g., a human actor, software application, or machine). An instance

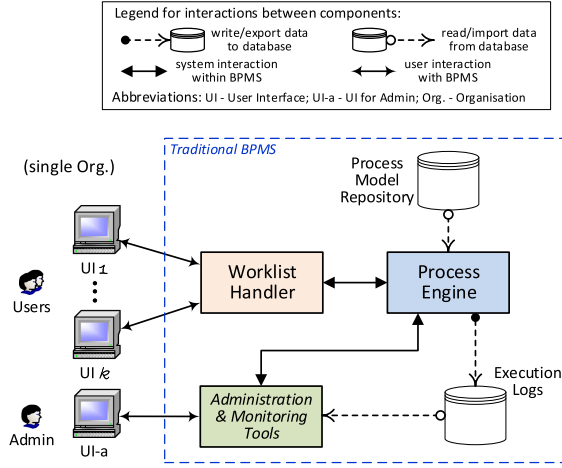


Fig. 2. Standard architecture of a traditional BPMS (adapted from [4]).

of process execution is often known as a case, and each case is uniquely identified. A case has a start and an end, and comprises a sequence of task executions. An instance of task execution is also referred to as a work item.

One of the key principles of BPMS is that business process executions are *stateful*. The notion of *stateful process execution* refers to the fact that once a case is initiated, every state of the execution associated with that case may be used to determine a future state of the case, and therefore all the states of that case execution need to be recorded in the order of the execution, step by step, from the initiation until completion of the case. Consider an example of a travel booking process that consists of registering travel itinerary, reserving flights, hotels, or rental cars, and handling payment tasks. A case of travel booking is launched when a traveller submits his/her itinerary, and can be uniquely identified by a booking reference number. Thanks to stateful execution of the process, for each travel booking in progress, it is then possible to inquire and retrieve the status of the booking, roll back and compensate if the booking is cancelled, and so on.

Figure 2 depicts the standard architecture of a traditional BPMS. It follows the *workflow reference model* [13] proposed by the **Workflow Management Coalition (WfMC)** and is well recognised as the principle for designing and deploying BPMS [4, 26] on premise and for a single organisation. The core component is the *process engine*, responsible for creating, running, and completing cases from initiation to completion (or termination). More specifically, the process engine should store the sequence of *process execution states* and compute the next enabled work item at each step of the execution based on the execution semantics of the process. If an engine must stop, all the running process instances (or cases) have to be migrated to another engine. This is known as *case migration*—a research topic on its own that has attracted research interests such as [17].

Next, work items are managed by the *worklist handler* through which end users (e.g., staff in an organisation) interact with their own list of work items, e.g., to check out a work item to start the work, or to check in a work item when the work is completed. Other operational matters concerned with BPMS, such as resource administration (e.g., managing the access control of individual end users) and process monitoring (e.g., tracking the progress of each running case), are taken care of by the *administration and monitoring tools*.

There are also two important data repositories. One is the *process model repository*. Before a process can be executed in the process engine, it needs to be defined in the form of a process model using a process modelling language. Process models are created using a modelling tool,

which is also part of BPMS but is *not* further considered in this article, since we are focusing on business process execution (rather than process modelling). Hence, we assume that a process model is always made available to the process engine from the process model depository. The second data repository stores records of the (step-by-step) execution of each process in the form of *execution logs*, which are a valuable input for process analysis and monitoring activities.

3.2 Cloud Computing

3.2.1 Scalability and Interoperability. The cloud computing paradigm is faced with 13 challenges according to the recently published manifesto for future generation cloud computing [2]. At the top of the list is *scalability and elasticity*. A scalable and elastic service provides operational capabilities to improve performance of cloud computing applications in a cost-effective way, and is crucial in achieving both Quality-of-Service and economic benefits as well as in supporting the creation of scalable global enterprise applications. However, designing mechanisms to achieve scalability and elasticity in the service provision in a given domain (such as BPMS) requires relevant knowledge and understanding of the context specific to the domain and is yet to be exploited.

As another posed challenge on the list, cloud interoperability remains an open issue given the fact that existing platforms still operate in silos and providers' efforts for integration usually target their own portfolio of services. To address such concern, it is important to take into account the principle of *interoperability* in an open cloud environment to ensure effectiveness of cloud computing as a public resource. To this end, systems are expected to be interoperable over a minimal set of community defined standards and should not rely on any specific system or platform to avoid vendor lock-in [14].

3.2.2 SaaS Maturity Model. Deploying BPMS in the cloud to concurrently provide execution resources to multiple organisations is a typical example of the more widely defined SaaS. An SaaS application is expected to be *scalable*, *configurable*, and *multi-tenant-efficient* [3]. Scaling an application means maximising concurrency to address an increasing workload; being multi-tenant-efficient means maximising the sharing of resources across tenants while still being able to differentiate and isolate data that belong to different tenants; and being configurable allows each tenant to configure the way an application appears and behaves to its users.

Figure 3 depicts a well-recognised SaaS maturity model [3]. It specifies four different methods for deploying SaaS, with each being assigned a particular level of maturity. Each level addresses an emphasis on one or more of the above three criteria for the design of an SaaS application. At *Level 1*, each tenant has its own customised instance of an application on the host's servers. This approach is easy to deploy but offers very few of the benefits of an SaaS solution. At *Level 2*, a separate instance of an application is hosted for each tenant, while all instances of the application use the same code implementation. Each tenant is provided with options to configure how the application looks and behaves to its users. At *Level 3*, a single instance is hosted that serves every tenant, with possible configuration of certain features, and provides resource sharing between tenants. The scalability of the application is, however, limited or non-existent. At *Level 4*, multiple identical instances of an application, made available via a load balancer, are hosted for multi-tenants.

3.3 Design Requirements

We use the term *BPMS for cloud* to refer to a BPMS solution that is designed for deployment in a cloud environment. Based on the relevant principles of BPMS and cloud computing described above, we draw three key requirements for the design of a solution of BPMS for cloud.

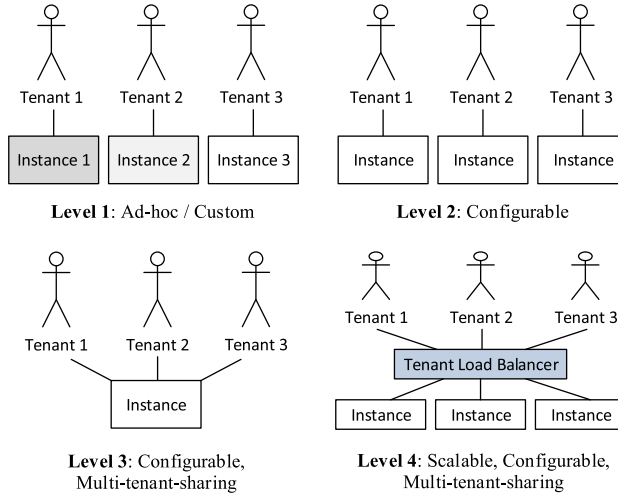


Fig. 3. Four-level SaaS maturity model (taken from [3]).

REQUIREMENT 1. A proposed design of BPMS for cloud should support **stateful** process execution, i.e., to keep track of the current execution state of each case and each of its work items from the beginning to the end of that case.

As discussed in Section 3.1, one of the key principles of BPMS is that *business process executions are stateful*. The principle must be followed in the proposed design of BPMS for cloud.

REQUIREMENT 2. A proposed design of BPMS for cloud should be **scalable**, i.e., capable of scaling **up and down** to address varying workload on demand.

As discussed in Section 3.2, scalability and elasticity is an important challenge to address in cloud computing, in particular, for proposing solutions related to a specific domain (e.g., BPMS). In this study, we define the notion of *scalability* as the ability to change in size or scale to cater to varying demands of workload (i.e., elasticity). Hence, a scalable BPMS for cloud is not only able to scale up to address increasing workload, but also to scale down to release underutilised (cloud) computing resources in the case of decreasing workload.

REQUIREMENT 3. A proposed design of BPMS for cloud should be **system- and platform-agnostic**, i.e., independent of any specific BPMS and any specific cloud platform in use.

Following the principle of *interoperability* discussed in Section 3.2, the proposed design of BPMS for cloud is expected to be interoperable and not rely on any specific system and platform.

4 DESIGN OF SOLUTION

4.1 System Architecture

To address **REQUIREMENT 3**, the design of system architecture for BPMS for cloud should be guided by relevant standards and/or reference frameworks in the domain. To this end, the WfMC's workflow reference model (Section 3.1) will be used to form the basis of BPMS architecture design, and the SaaS maturity model (Section 3.2.2) serves as an underlying principle to guide the re-design of traditional BPMS architecture for cloud-readiness.

Recall the three main components in the standard architecture of a traditional BPMS (Figure 2): the process engine, worklist handler, and administration and monitoring tools. For the design of

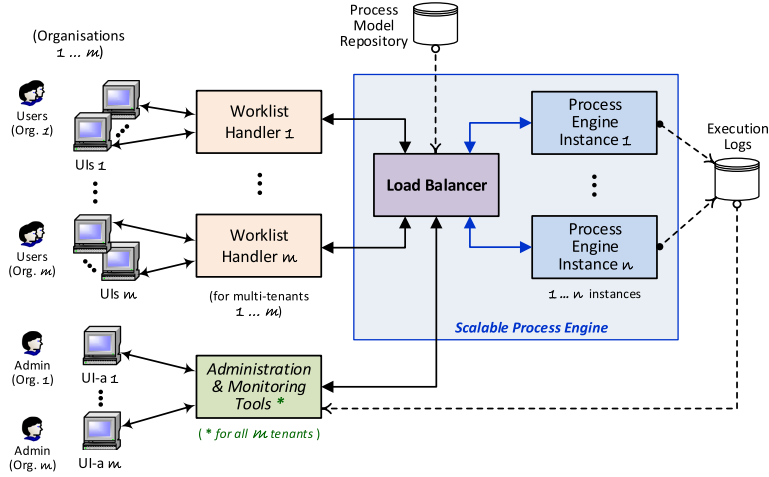


Fig. 4. Overall architecture of BPMS for cloud.

BPMS for cloud, we examine respectively the requirements of each of the three components in a cloud environment, and as a result, the re-design of each of the components is proposed by adopting the design principles at different levels of SaaS maturity model accordingly.

Figure 4 depicts the resultant design of an overall architecture of a BPMS that operates for multiple organisations in a multi-tenanted cloud environment.

Firstly, there are multiple instances of worklist handler. Each of them is configured for an individual organisation to best capture their specific requirements regarding the management of their work items (e.g., how a list of work items is presented to an end user, how such a user interacts with his/her work list). This is designed according to *Level 2* of the SaaS maturity model, since *configurability* by each organisation is the key concern to address.

Secondly, there is a single instance of administration and monitoring tools shared by multiple organisations. This is based on *Level 3* of the SaaS maturity model, for the reason that *sharing* the functions to support operational matters (e.g., process tracking and monitoring) between the administrators of different organisations is the main focus. Note that scalability is not a key concern for this component since handling operational matters usually has a light workload. Furthermore, the *Level 3* of the SaaS maturity model serves as a better strategy to address security and privacy concerns in such setting.

Finally, the concept of a *scalable process engine* is introduced to support the enactment of *multiple instances* of process engines via the use of a new component called the *load balancer*. This is designed according to *Level 4* of the SaaS maturity model, so that the process engines can address a large amount of simultaneous requests from more than one organisation, and at the same time it allows the sharing of multiple process engines among the organisations.

4.2 Load Balancer

The design of a scalable process engine addresses **REQUIREMENT 2** via its key component—load balancer. As Figure 4 depicts, a load balancer is positioned as a middleware layer between multiple engines and multiple worklist handlers. Upon receiving requests from multiple tenants (via their worklist handlers), the load balancer distributes or directs each request to one of the active engines, and upon receiving responses from the engines it directs each back to the corresponding tenant.

The load balancer is designed to fulfil two major responsibilities. Firstly, each active engine in the cloud should have an *even* distribution of workload relative to that of the other engines.

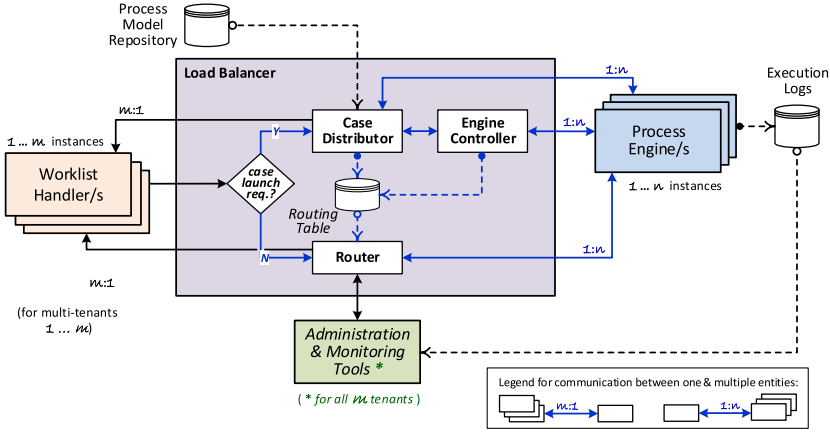


Fig. 5. Structure of a load balancer in a scalable process engine.

The main objective is to meet **service level agreements (SLAs)** for metrics such as acceptable response times and service availability (or uptime). It is important that the system responds in similar time frames to those of (standalone) on-premise BPMS deployments and the service is available without interruption. Secondly, to ensure that no engine is over- or underloaded, the load balancer is expected to control the capacity of multiple engines to cater to the demand from the multiple tenants. The main objective is to achieve scalability—the ability to scale up and scale down the engines’ capacity given the workload at each specific time.

Figure 5 illustrates the proposed design of a load balancer. It comprises a *case distributor*, an *engine controller*, a *router*, and a *routing table*, and they work together to support the above functionalities.

The case distributor is responsible for distributing a case to one of the process engines upon receiving a case launch request. To address REQUIREMENT 1, once a case is allocated to an engine, it will be executed in a *stateful* manner, meaning that all the work items belonging to the case will be handled by the same process engine until the completion of the case. After a case is launched, the router takes over the responsibility for communication and directs work items between the corresponding worklist handler and process engine.

The engine controller is designed to support *scalability*. It is responsible for the following tasks: (a) monitoring the engines’ *busyness status* (which is a workload metric; see its definition in Section 4.3), (b) starting up a new engine in the need of scaling up, (c) closing down a running engine in the need of scaling down, and (d) performing case migration as part of task (c).

The routing table records the information necessary in coordinating each work item between the right process engine and the right worklist handler. The case distributor and engine controller both have write access to the routing table, whereas the router can only read from the table.

4.3 Data Specification

A scalable process engine mainly handles data related to process execution. To specify the key data handled by the engine, we use **object-role modeling (ORM)** [9], a conceptual, fact-oriented modelling language for data specification. Figure 6 depicts an ORM model specifying the key data associated with a scalable process engine in our design.

Most of the concepts, such as process, case, task, work item, (workflow) engine, and worklist handler, as well as how they relate to one another, were introduced in Section 3.1. The notion of *session* is new in the ORM model. According to the design of the system architecture (Figure 4), each

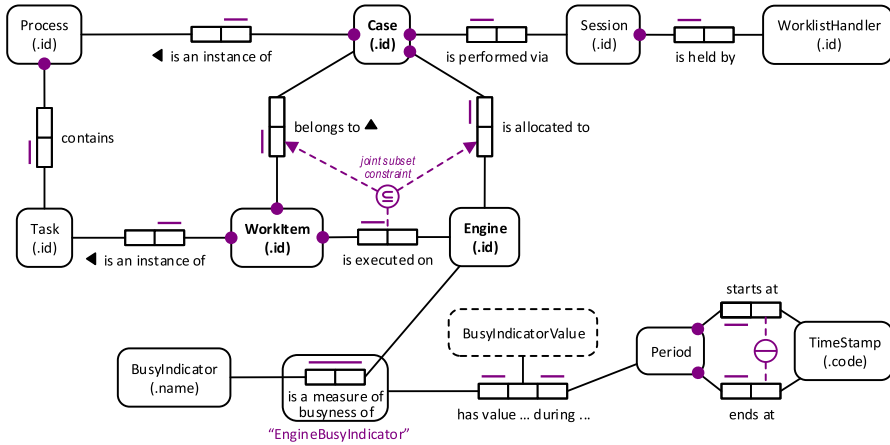


Fig. 6. An ORM model conceptualising the key data of a scalable process engine.

multi-tenant organisation hosts a worklist handler, a component that controls the distribution and management of work items between the cloud BPMS and the individual user work queues (i.e., the worklists) of the organisation. Before its work can begin, each worklist handler must first securely authenticate with the cloud BPMS and, if successful, it will receive a session handle, which acts as both a unique identifier and a secure token (i.e., proof of authentication). The session handle is then passed with the parameters of each request from the worklist handler to the cloud BPMS, where it serves to signify the secure authentication of the worklist handler. It is also used by the case distributor component of the cloud BPMS as an identifier of which worklist handler has made the request, and so to which the reply must be sent. For example, a worklist handler will send its session handle along with the other relevant parameters of a work item check-out request, and the case distributor component will use the session handle to identify the worklist handler to return the checked-out work item to.

Also, we introduce a *busyness indicator*, a performance measure of an engine that quantifies how busy it is (i.e., how heavily utilised). During process execution, the busyness of an engine changes over time. This dynamic is captured by the fact that an engine's busyness indicator has a specific value given a certain time period which is uniquely identified by its start and end timestamps.

It is interesting to mention that the joint subset constraint in our ORM model is used to declare that if a work item is executed on an engine, then that work item must belong to a case that is also allocated to that engine. This precisely captures [REQUIREMENT 1](#) on *stateful* process execution.

4.4 Component Interactions

A further important design consideration is the specification of key interactions between the different components of a scalable process engine. We use **unified modelling language (UML)** sequence diagrams [15] to specify how these components work with each other to support case execution, and to achieve scalability.

4.4.1 Case Execution. Figure 7 depicts a UML sequence diagram that captures the interactions between worklist handler, case distributor, engine controller, router, and multiple process engines for executing a case. In general, a case execution consists of five phases: (a) to launch a case, (b) to compute the next (enabled) work item, (c) to check out the work item (begin work), (d) to check in the work item (complete work), then to repeat the phases (b), (c), and (d) until there are no more enabled work items, and finally (e) to complete the case.

Upon receiving a request to launch a new case in phase (a), the case distributor requests from the engine controller the current busyness status of all available process engines. The engine controller communicates with each engine to collect its busyness data within a certain time window, and afterwards it responds to the case distributor with the busyness status of all the engines. The case distributor then uses that data to select the most appropriate engine, and communicates with that engine to launch the case. Note that the decision about which engine to select is based on a specific *load distribution strategy* used by the case distributor when different load distribution capability levels are defined (see Section 4.5). After the case is launched on the selected process engine, the case distributor records into the routing table the information necessary for correctly routing the work items belonging to the case.

Once the case is launched, the router takes over the responsibility of communicating between the engine and worklist handler. In phase (b), the engine computes the next enabled work item and the router passes it to the worklist handler. The work item is checked out in phase (c), performed by the relevant end user or application, and then checked in upon its completion in phase (d). Phases (b)–(d) will be repeated until there are no more enabled work items, and then the case will be completed in phase (e). As such, the UML sequence diagram in Figure 7 also demonstrates the support for stateful process execution in our design.

4.4.2 Scale-Up Scenario. Figure 8 depicts a UML sequence diagram for the design of starting a new engine instance in a scale-up scenario. In the beginning of the case launch procedure, the engine controller checks with each process engine to get its busyness status, upon request from the case distributor. The data collected may indicate that overall the engines are at or close to their maximal load (i.e., beyond which SLA metrics may be compromised), which requires scaling up the resource capacity. As such, the engine controller will start a new engine, add the engine information into the routing table, and then send back the busyness status of all the active engines, including the new engine, to the case distributor. Afterwards, the case will most likely be launched on this new engine (due to its more available capacity than that of others).

4.4.3 Scale-Down Scenario. Figure 9 depicts a UML sequence diagram for the design of closing down a running engine in a scale-down scenario. Independent of case execution activities, the engine controller monitors the busyness status of each active engine at regular time intervals. If it finds that overall the engines are underloaded (e.g., only 50% of the capacity is utilised in each engine), the engine controller will work to scale down the current available engine capacity. It will do so by selecting one of the running engines to close down (e.g., one with the lowest workload at the time), instruct to suspend all the ongoing cases on that engine, and deactivate the engine. Next, the engine controller will communicate with the case distributor, asking for a schedule of re-allocating the suspended cases to the remaining active engines. For each suspended case, the engine controller will then perform case migration from the deactivated engine to an active engine specified in the above schedule of case re-allocation, update the engine information for the case in the routing table, and resume the case on the engine to which it is reallocated. Upon completion of all the above operations, the engine controller will finally shut down the deactivated engine.

4.5 Load Distribution Capability

A load balancer is in charge of controlling the distribution of the varying load of process instance execution to multiple process engines at runtime. This is achieved by applying load distribution strategies. Recall the UML sequence diagram of case execution in Figure 7. Load distribution strategies are deployed within the case distributor of a load balancer when selecting an appropriate

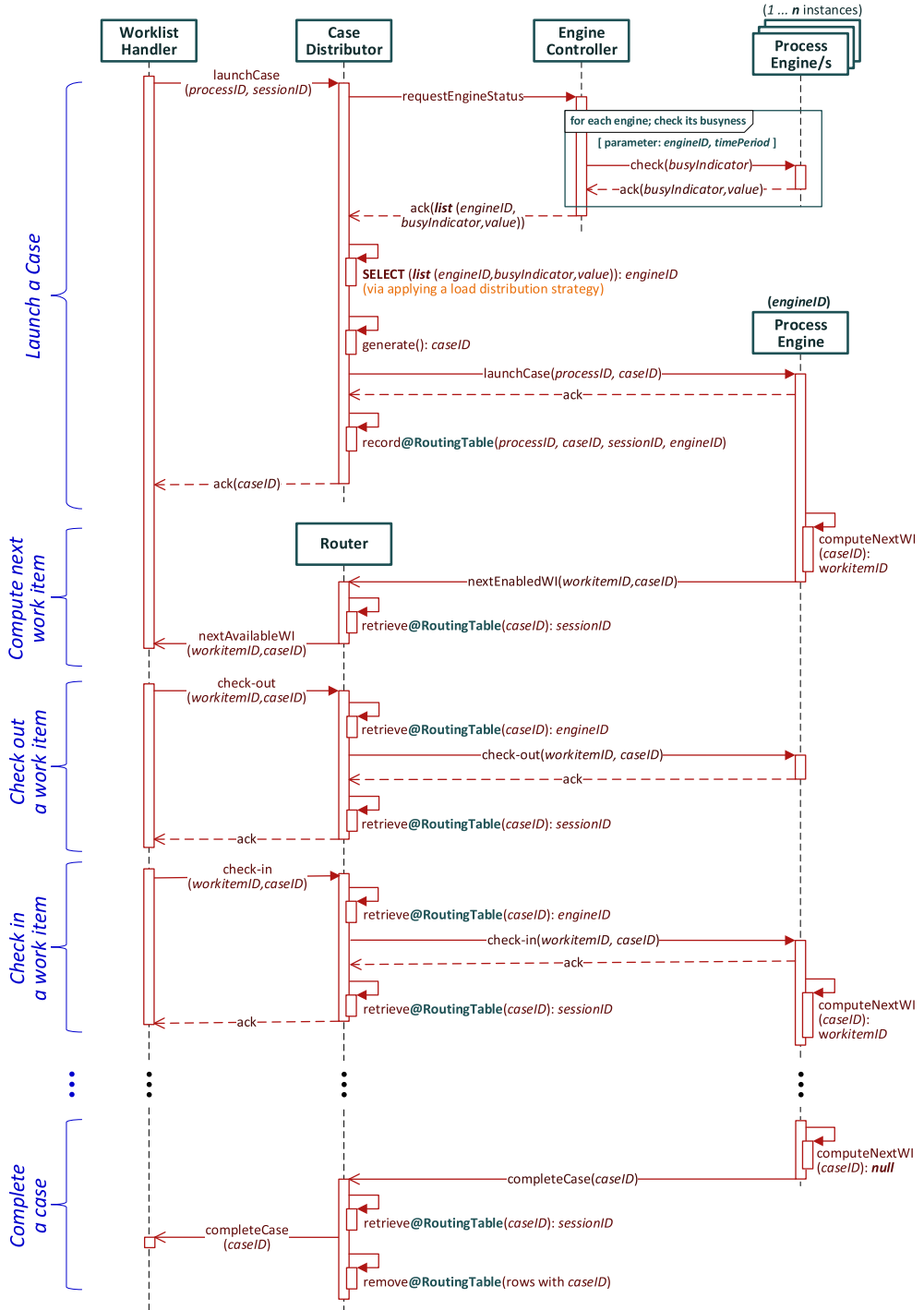


Fig. 7. A UML sequence diagram capturing the component interactions for a case execution.

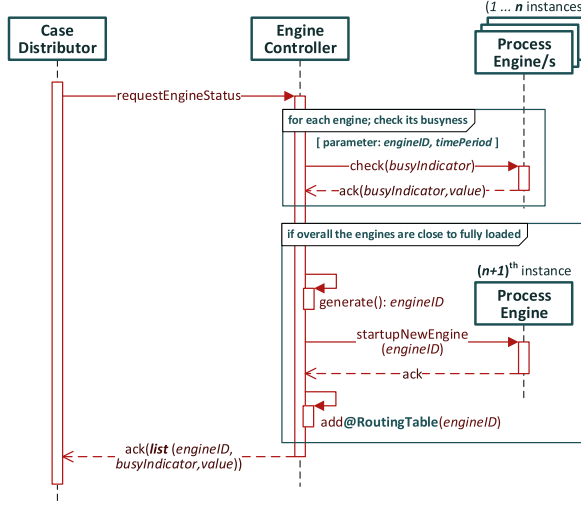


Fig. 8. Scale-up scenario: Start a new engine.

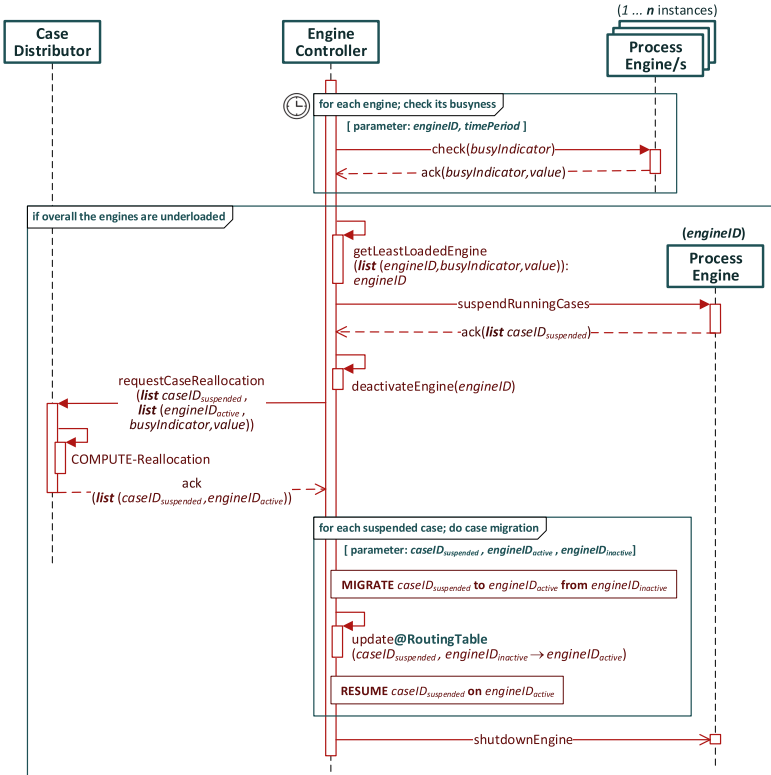


Fig. 9. Scale-down scenario: Shut down a running engine.

engine to run the case. By applying various load distribution strategies, a load balancer can achieve different levels of capability in load distribution. To aim for an even (i.e., balanced) distribution of load across all active engines, we consider that a load balancer in BPMS for cloud can be supported through the dynamic calculation of the busyness of each engine using different strategies, leading to four capability levels as defined below.

- *Capability Level 0*: Apply a general work scheduling or resource allocation mechanism (such as random choice or round robin) *without* considering how busy each engine is. This is essentially equivalent to a default stateless load distribution.
- *Capability Level 1*: Consider each engine's busyness *at the exact instant of time* when a new case is required to be distributed to an engine for execution.
- *Capability Level 2*: Consider each engine's busyness *within the time period* of a sliding window looking *backward* (i.e., several time units in the past) from the time of case distribution.
- *Capability Level 3*: Consider each engine's busyness *within the time period* of a sliding window looking *forward* (i.e., several time units in the future) from the time of case distribution.

Discussions. With regard to load distribution in the above design of BPMS for cloud, two relevant aspects are worthy of discussion. One is concerned with SLA and the other relates to case complexity.

While the basis of any SaaS SLA revolves around the metrics of service uptime and responsiveness, it is recognised that certain other metrics are often sought. For example, organisations may have a need to prioritise their cases, where cases of a particular specification are deemed to be more critical than others and as such should be assigned a priority. Such prioritisations, on the surface, would incur the expectation of even faster response times than usual for higher over lower priority cases, achieved through the suspension of cases of lower priority to make way for critical cases. However, this ignores the fact that a cloud-based BPMS has the ability to upscale capacity at any time, and will do so to accommodate new critical cases whenever necessary, such that the performance of those cases will not suffer in relation to those of lower priority running on other engines.

Further, the ability to prioritise individual cases is typically a function of the process engine itself, rather than any load balancer that distributes the work based on load. So, fundamentally, the engine would deal with work prioritisation rather than the load balancer *per se*. Nevertheless, the effects of case priorities on workload and work balancing forms one direction for future work.

Next, it is recognised that while some process specifications are quite trivial, comprising a small number of tasks, others may be very complex and/or comprise many tasks. It is natural to assume that the complexity of a process would represent an important consideration in any calculation of process engine workload; however, in actuality, case complexity has no real effect on engine workload. To understand this requires an understanding of fundamental process engine functionality. More specifically, once a case is assigned to a particular engine, the engine is responsible primarily for two things: (i) to announce the next available task in a process, according to its control flow; and (ii) to action the completion of a task, which begins determination of the next task (if any) to announce, according to control flow. Tasks are announced to resources external to the engine (i.e., worklist handlers, applications, services) so that the relevant resource undertakes the work of the task to be performed. Essentially, this means the engine is oblivious to the complexity of the case, rather it deals with control flow at *the task level*, and so is aware only of the last completed task and the next one. Furthermore, since the engine is to delegate work to external resources, the complexity of the actual work required to perform the task has no impact on engine workload and/or performance.

A second form of case complexity takes the form of a specification with a long sequence of very short-lived tasks. Such a specification would impinge on the workload of an engine, for the reasons outlined above, and the number of task starts and completions per time unit forms part of the current busyness factor calculation. However, the ability to upscale and downscale engines means that such specifications could be moved to lower utilised engines, or new engines could be “spun up,” in an ad-hoc manner to accommodate such bursty activity. Nevertheless, the only way to have runtime awareness of the behaviours of processes is to have the ability to interrogate the histories of previous cases of a process and to use that data to calculate the expected impact of a case on workloads. This aspect forms another direction for future work.

5 SYSTEM REALISATION

A prototype of a scalable BPMS that implements the conceptual design proposed in Section 4 has been realised as a new discrete component within the YAWL environment,¹ which is open source, stable, and offers a service-oriented architecture. It allows the new load balancer component to be deployed independent to the existing components. Importantly, no changes were required to be made to the YAWL environment itself to enable support for a load-balanced array of multiple YAWL (process) engines in the cloud. However, to support the up- and downscaling functionality, a small extension was required to be added to the YAWL engine to support the dynamic import and export of process instances. This section focuses on the implementation of the *load balancer* component (Section 5.1) and several load distribution strategies to meet different capability levels (Section 5.2) in our prototype.

5.1 Prototype of Load Balancer

The prototype of a load balancer has been developed so that it is extensively configurable and changes to configuration values can be applied in real time. The load balancer component oversees a set of available YAWL engines. It intercepts all interface calls from external worklist endpoints and redirects them to the appropriate engine for processing. This is achieved by a trivial configuration change in each worklist so that rather than having a single engine as its remote endpoint, that endpoint directly connects to the load balancer component. This way the perspective of a user is unchanged; it will appear as though they are conversing with a single engine.

Whenever the load balancer component receives a request from a worklist or administration interface to launch a new process instance, it will poll each engine for its current busyness status, captured by, e.g., an *engine busyness measure* (see Section 5.2), according to the pre-configured load distribution capability level. Next, it will pass the request to the least busy engine, except when configured to capability level 0 where a choice is made without considering engines’ busyness status. Recall that a main objective of the load distribution (set in the scope of this research) is to make sure that each active engine in the cloud array has an equitable load to that of the other engines in the array. Hence, the load balancer component applies a load distribution strategy to select the *idlest* (i.e., least busy) engine, based on the currently configured capability level. To this end, our prototype currently supports all four load distribution capability levels specified in Section 4.5, which means that a different method is used to find the idlest engine at each capability level on a case launch request, as detailed in Section 5.2.

Another important functionality for load balancer to fulfil is scalability. Based on our conceptual design, this is where the *engine controller* sub-component of a load balancer also comes into play. On the one hand, if, upon a launch case request, the load balancer reports that all engines are at or near their maximal (configurable) loads, it will request the engine controller to bring a new engine

¹<https://github.com/yawlfoundation/yawl>.

online (*scale up*), and then will launch the new instance on that engine. On the other hand, the engine controller polls each of the n available engines at regular (configurable) time intervals for its current busyness status. If it finds that the sum of busyness factors can be redistributed over $n - 1$ engines and have those remaining engines continue to operate beneath maximal load, it will trigger the procedure to *scale down* the engine with the lowest current load.

In addition, specific measures are taken across several time units so that unnecessary “churn-ing” is avoided (i.e., continual scaling up and down). In doing so, the load balancer and its engine controller sub-component maintain best use of available (computing) resources, and ensure that no resources (in the form of engines) are underutilised.

5.2 Implementation of Load Distribution Strategies

5.2.1 Engine Busyness Measure. We describe two important concepts relevant to runtime process execution. Firstly, a call to launch a new case would form a *request*, as would one to complete a work item or begin a new work item. A request is a programmatic call to an endpoint of an **Application Programming Interface (API)**, together with any required data values and parameters. For example, a load balancer provides an API for communications with a worklist handler. Since each request must be serviced, the more requests an object (e.g., a load balancer or a process engine) receives through its API per time unit, the busier that object is. Secondly, a load balancer and a process engine are hosted on a server running one or more *threads*. A thread, at the computing level, is a single flow of control within a program. Since servers have access to multiple CPUs, they can accommodate a number of threads simultaneously, each potentially performing a different part of a program simultaneously. Generally speaking, a server will create more threads when under load. As such, the current thread count of a server, when compared to its count at rest, is a good indicator of how busy a server is overall.

We consider that a process engine’s busyness at runtime is mainly determined by four key metrics related to its operational environment. These are the following:

- *Number of requests per second* (denoted by R): The number of requests (associated with process execution) received and needed to be processed per second by an engine;
- *Processing time per request* (P): The average processing time (in milliseconds) per processed request by an engine;
- *Number of work items per second* (I): The number of both work items checked-in to an engine and work items checked-out from that engine per second; and
- *Number of active worker threads* (T): The number of worker threads currently executing in an engine (or more precisely in the engine’s container).

Given the fact that each engine has a work capacity, it is necessary to capture an upper limit of each relevant metrics (as the above) that represents a maximum *comfortable* load for an engine. Accordingly, we define four *up-limit parameters* and all are configurable parameters to account for different hardware infrastructures. These are the following:

- A maximum number of requests an engine is capable to process per second (L_R);
- An upper bound of the average processing time per request by an engine (L_P);
- A maximum number of both work items checked-in to an engine and work items checked-out from the same engine per second (L_I); and
- A maximum number of active worker threads that can be managed by an engine (L_T).

Next, each of the four metrics (R , P , I , and T) can have a varying impact on the busyness of a process engine. This may be due to the differences in the hardware of different operating environments. For example, the number of threads a system can effectively create is limited by the

ALGORITHM 1: Selection of Idlest Engine

```

1: procedure IDLESTENGINE(mode, eSet)
2:   if mode.random then                                ▶ Baseline random distribution
3:     return RANDOMITEM(eSet)
4:   end if
5:   idlest ← nil
6:   lowb ← max                                           ▶ Prime lowest busyness score
7:   for all e in eSet do                                ▶ For each engine
8:     b ← GETBUSYNESS(mode, e)                         ▶ Get engine's busyness score
9:     if b < lowb then
10:      b ← lowb
11:      idlest ← e                                       ▶ This e refers to current idlest engine
12:    end if
13:  end for
14:  return idlest
15: end procedure

```

number of CPUs available to it. The number of threads that can be handled per time unit is affected by memory resources. The number of work item creations and completions per time unit will differ depending on whether processes are generally short-lived and bursty, or long-lived and infrequent, or a combination of those factors. It also may be an SLA requirement to place more importance on processing time, which would mean a higher weight for that metric; and processing time is heavily affected by available computing resources. Hence, it is important to have the ability to assign weights to our four metrics. As such, we specify four *weighting parameters*, W_R , W_P , W_I , and W_T , for these four metrics, respectively. These are also configurable parameters and weightings can be set for each so that an accurate factor for any operating platform can be established.

Based on the above, we define the following engine busyness measure (B) for computing a busyness score to indicate (quantitatively) how busy an engine is at a given point of time or for a given time period.

$$B = \frac{(R/L_R) \cdot W_R + (P/L_P) \cdot W_P + (I/L_I) \cdot W_I + (T/L_T) \cdot W_T}{4} \times 100.$$

5.2.2 Idlest Engine Selection. On receiving a request to launch a new case, the load balancer component uses a selection algorithm to choose the *idlest* engine, based on the currently configured load distribution capability level. The method for engine selection upon case launch is specified in Algorithm 1. The procedure begins with the current capability level (*mode*) and the set of active engines (*eSet*) (line 1). A reference to the current least busy engine is stored in the *idlest* attribute (line 5), and the lowest busyness score is stored in the *lowb* attribute (primed on line 6). Each engine's busyness score is computed, depending on the current capability level (line 8). If it is less than the current value of the *lowb* attribute, then that busyness score is stored in *lowb* (line 10) and a reference to that engine is stored in the *idlest* attribute (line 11). Once all engines have been processed, the idlest engine reference is returned.

5.2.3 Load Distribution Strategies. A different method is used to find the idlest engine at each capability level on a case launch request. Several load distribution strategies to respectively achieve the conceptual load distribution capability levels 0–3 (specified in Section 4.5) have been implemented in the load balancer component. Which capability level to apply is set via a configurable

parameter, and depending on the level set, one of the following four procedures is used to allocate new process instances to engines for each case launch request.

Capability Level 0. A random number, limited by the number of available engines, is generated in the programming environment and then used to select an engine from the set, and that engine is allocated the new instance. Thus, a random distribution of cases is achieved, and no reference is made to the current busyness status of each engine.

Capability Level 1. Each engine is immediately polled for its busyness measure at a given point of time. The engine with the lowest reported busyness score (i.e., the least busy engine at that point of time) is allocated the new instance.

Capability Level 2. A backward sliding window to smooth factor readings is applied. The method uses an exponentially weighted moving average to calculate engine busyness, which allows for a configurable weighting factor to be applied in an exponentially decreasing manner to a cache of immediately prior readings. It is calculated recursively as follows:

$$S_t = \begin{cases} Y_1, & t = 1 \\ \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1}, & t > 1, \end{cases}$$

where

- α is the weighting factor (a.k.a. the forget factor), between 0 and 1, representing the degree of decrease for older readings, which diminish more quickly as values of α approach 1.
- Y_t is the engine busyness reading at (point of) time t .
- S_t is the weighted moving average at (point of) time t .

This procedure provides for previous busyness scores of an engine to have an influence on the current busyness score so that some account can be made of history and non-characteristic spikes are smoothed, rather than the simple snapshot of the engine's busyness used in capability level 1, when such an instant's measure reading may not necessarily be indicative of the engine's busyness.

Capability Level 3. One of three algorithms is dynamically invoked, depending on the range of cached prior busyness scores stored as a time series, and attempts to predict near-term future values of the series. The three algorithms used are single variable regression, single variable polynomial regression, and multiple variable linear regression.² Fundamentally, each algorithm seeks to extrapolate from the previous set of busyness scores for each engine a trend line or curve that best fits the data, and then uses that trend to forecast upcoming busyness scores. If a threshold of time series data values has not yet been met, this strategy reverts to capability level 2.

6 PROOF-OF-CONCEPT EVALUATION

A set of experiments were devised and performed to evaluate the capabilities of the prototype's scale-up and scale-down functionality. Details of the design and results of the experiments are discussed in this section.

6.1 Design of Experiments

The load balancer component and a total of four YAWL engines were installed on the QUT High Performance Computing platform,³ using a discrete **virtual machine (VM)** for each engine, and

²The Java-based OpenForecast libraries are used to dynamically select the best-fit algorithm and to perform the relevant calculations (see www.stevengould.org/software/openforecast/).

³<https://www.qut.edu.au/about/contact/h/high-performance-computing>.

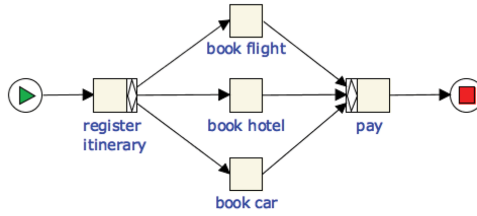


Fig. 10. Process specification used for simulation (adapted from [23]).

another for the load balancing component. Each VM ran on a single core of an Intel Xeon CPU E5-2680 @ 2.70 GHz, and 4 Gb of RAM was available to each. This configuration provided an effective simulation of a *cloud-based platform* for the purposes of the evaluation.

Since the *scaling* functionality is to be evaluated, when selecting a process for the experiments, we are not necessarily interested in those processes of a complex nature; instead a simple process that can facilitate the simulation of varying workload would suit the purposes of the evaluation. Figure 10 depicts the *process specification* used for the experiments. It is based on the example found in [23], which models a simplified travel booking process. After a travel itinerary is registered, bookings of hotels, flights, and/or rental cars can be made according to the traveller’s needs, and finally payment is made. The process is specified using YAWL notation, where the two circles signal respectively the start and end of the process and each square represents a task in the process. A diamond-shaped notation attached to a task indicates an (inclusive) OR construct. For example, an OR-split has more than one outgoing branch specifying multiple choices and it is used to model an inclusive decision-making. Given the travel booking process in Figure 10, an XQuery function was applied to each of the OR-split predicates following the “register itinerary” task so that for each instance of the specification, one, two, or all three of the subsequent tasks were executed. This arrangement allowed for some differences between process instances, simulating actual choices that would be made by human participants on a case-by-case basis in ordinary circumstances.

A bespoke *simulation tool* was used to launch a configured number of new instances of the given process specification. The tool supports the entire execution of each case by processing work items with resource “robots,” that is, automated agents that mimic the role of human resources. These agents were configured to simulate processing of each work item for a randomised period within configured limits. For the *load distribution strategy*, capability level 3 was chosen since it has been found to give the best results in terms of evenly distributed load between engines.⁴

Two sets of experiments were devised to compare performance when scale-up or scale-down functionality was enabled and disabled (achieved via a setting in the configuration file). The first set measured the effect of intense load on the busyness of the engine array, first with scale-up functionality disabled, and again when enabled. The second set observed the effect of low load on the busyness of the engine array, with scale-down functionality disabled and enabled, respectively.

Table 1 lists the set of configuration parameters applied for each experiment. Configuring an equal minimum and maximum number of engines effectively disables the scale-up and -down functionality. For the experiments testing the scale-up functionality, requests to launch new process instances are created by the simulation tool every half second, and the resource “robots” are set to process tasks very quickly (0.2–0.8 seconds), which causes high workloads on the engines as tasks are completed and immediately subsequent tasks started. For the experiments testing the scale-down functionality, requests to launch new instances are created every 2 seconds, and the “robots”

⁴A detailed account of the evaluation of the various capability levels can be found in our previous works (*reference removed for blinded review*).

Table 1. Configuration of Simulation Parameters

	Min # Engines	Max # Engines	# Instances	Launch Interval (msec)	Min Task Processing (msec)	Max Task Processing (msec)
No scale up	2	2	500	500	200	800
Scale up	2	4	500	500	200	800
No scale down	4	4	300	2,000	2,000	5,000
Scale down	2	4	300	2,000	2,000	5,000

Table 2. Statistical Summary of Simulation Results

	No Scale Up	Scale Up	No Scale Down	Scale Down
Min # Engines	2	2	4	2
Max # Engines	2	4	4	4
2 Engines (time %)	100	12.3	–	17.2
3 Engines (time %)	–	29.1	–	33.1
4 Engines (time %)	–	58.6	100	49.7
Min Busyness Score	4.3	4.2	3.3	3.3
Max Busyness Score	172.2	108.8	82.7	103.2
Mean Busyness Score	122.3	48.3	36.2	45.3
Min Response Time (msec)	39	40	38	38
Max Response Time (msec)	267	106.3	66	98
Mean Response Time (msec)	154.1	57.7	50.8	59.0
SLA Violation (%)	83.2	4.5	0	3.2

are set to process tasks relatively slowly and with far greater range of duration (2–5 seconds), which will cause only low workloads on the engines.

6.2 Simulation Results

The results of the experiments are shown in Figure 11 and summarised in Table 2. For each chart in Figure 11, the y -axis depicts the busyness score/response time (milliseconds), while the x -axis depicts time units. The grey bars indicate the range of busyness scores across all running engines for each time unit, and the overlaid dark line indicates the average response rate across all running engines per time unit.

For reference, a reasonable response time for a typical service level agreement would be less than 100 milliseconds; that is, any response time greater than 100 milliseconds may be considered an SLA violation. A busyness score of 100 is the upper limit for a “comfortable” engine; that is, a score less than 100 means the engine is within capacity and is unlikely to cause an SLA violation, while a score greater than 100 indicates an engine running higher than peak capacity. A violation is likely whenever the busyness score for an engine exceeds 100.

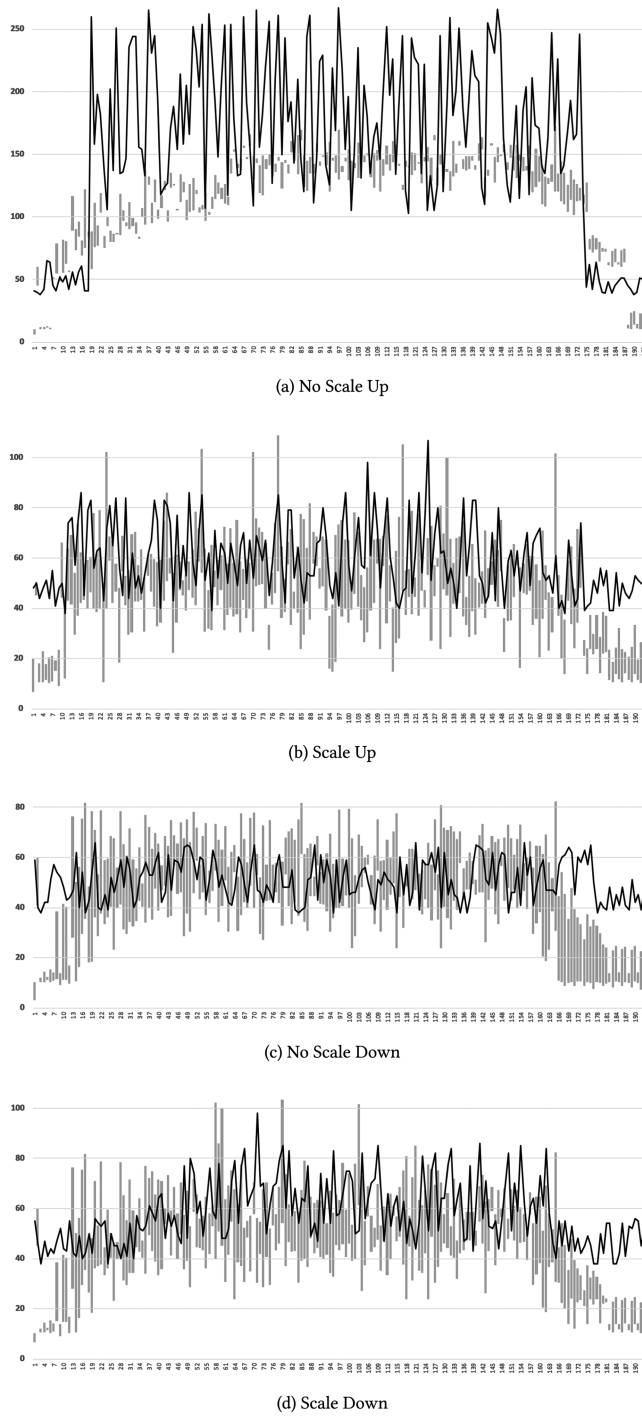


Fig. 11. Graphical visualisation of simulation results from each of the four experiments.

6.2.1 Scale-Up Scenario. When scale-up functionality is disabled (Figure 11(a)), under heavy workload the results indicate that while the load remains relatively uniform across the two available engines (shorter grey bars), they soon become overwhelmed and the response time SLA is exceeded for the major part of the exercise. Note that for this chart, the scale of the y -axis is double that of the other three charts in Figure 11. It is evident that the two engines have exceeded maximal load in terms of the SLA early in the experiment and have little chance of recovering during the remaining time until some time after the last process instance has been launched.

On enabling the scale-up functionality with a minimum of two and a maximum of four engines (Figure 11(b)), the heavy workload soon causes the scaling up of the free engines so that the load is able to be spread more successfully and evenly across the engine array. The SLA violation period is much less, but not nil; there still remains a small amount of time during the experiment when the SLA is violated. Some of these violations correspond with a new engine coming online, which as expected would account for higher busyness scores and response times immediately before and during these events. However, the fact that a small number of SLA violations were experienced is an indication that even four engines were not enough to handle the peak load of the simulation, but the results do allude to the fact that having a fifth engine available would have most likely fully erased the risk of violation.

For this experiment, there was a hard upper limit of four engines. However, in an environment of potentially unlimited VMs hosting process engines, which would be available if using a third party supplier of cloud-based platforms such as **Amazon Web Services (AWS)**⁵ or Google Cloud Platform.⁶ The scale-up functionality would continue to bring new engines into the array whenever all current engines were at maximal load, so that SLA violations could be avoided completely. Once peak load had passed, the VMs could be handed back to the platform provider using the scale-down functionality discussed next. Some cloud platform providers have plans that charge by the second, which makes this approach very convenient, as VMs can be “rented” for only as long as needed to meet load, thus making the most efficient use of resources.

6.2.2 Scale-Down Scenario. With scale-down functionality disabled (Figure 11(c)), that is, making all four engines in the array available for use at all times, SLAs were never violated during the experiment (as expected). Compared to Figure 11(b), it can be seen that both response rates and overall busyness scores remained at a relatively constant and low level, indicating that the light load imposed did not cause any stress when spread across four engines. However, this approach represents a serious waste of resources, and it is evident from the results that the workload could have been consolidated across fewer engines without risking any violations.

When scale down was enabled and the experiment rerun (Figure 11(d)), the functionality worked quite well to remove active, underutilised engines from the array and manage the load with those remaining. There were small lags of between 0.22 and 0.28 seconds when each of the two engines was scaled down and cases transferred to the remaining engines, and thus a very small period of SLA violation. Nevertheless, the export/import process instance algorithms worked quite efficiently so the only momentary delays would be noticed by end users, if at all. And that small period of violation was more than offset by the amount of time during the experiment when one, and then two, of the engines were removed from resources underutilisation, thus representing appreciable savings in platform hire costs.

6.2.3 Comparison with Single Engine. The results of running the same tests on a single engine (i.e., without the involvement of the load-balancing component) are shown in Figure 12. The

⁵<https://aws.amazon.com/>.

⁶<https://cloud.google.com/>.

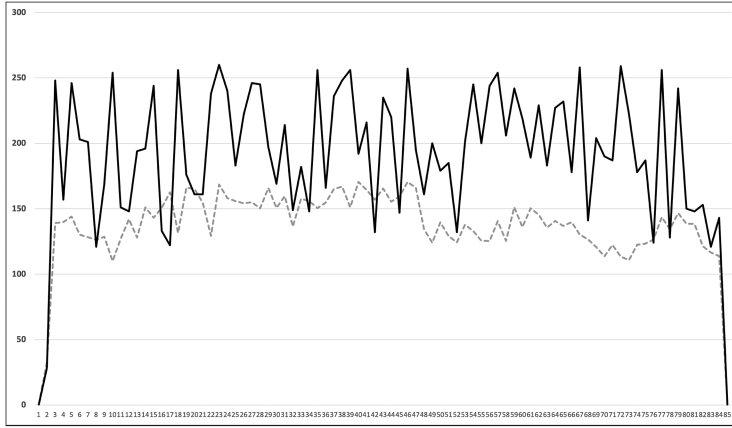


Fig. 12. Graphical visualisation of simulation results when using a single engine.

same configuration settings, process specification, and workload as the no-scale-up experiment (cf. Table 2 and Figure 10(a)) were used here.

It is apparent that the outcome of the single engine experiment shares some similarities with the no-scale-up results. The busyness factor of the engine, shown as the dotted line in Figure 12, consistently exceeds 100, meaning the engine was beyond comfortable peak load for the duration of the experiment. Additionally, the response times never dipped below 118 milliseconds, indicating the engine could not meet agreed SLA requirements for the entire life of the experiment. That is, the SLA was violated for 100% of the experiment. Even though the workload exceeded normal operational expectations, it is clear that there may exist workloads that a single engine cannot be expected to comfortably handle, further demonstrating the advantages of the approach involving several engines in a load-balanced environment.

There are two event types where the load balancer incurs additional overheads when compared to the operations of a single engine:

- (1) **On starting a new case:** When a new case is launched, the load balancer must first determine which of the available engines is the appropriate one to manage the case. Here, the load balancer queries each engine for its busyness factor, and then forwards the case launch request to the selected engine. In our experiments, the algorithm for selecting and recording the appropriate engine took 6–8 milliseconds. Forwarding a request to the selected engine took <2 milliseconds. In all, an operation of 6–10 milliseconds made very little impact on the overall request-response cycle.
- (2) **All other requests:** Besides a new case launch, all other requests are simply forwarded by the load balancer to the appropriate engine. A hashed look-up table of case-identifier $\longleftrightarrow$ engine-identifier pairs is kept in memory, ensuring a value search is almost instantaneous. Then, the forwarding of the request again took <2 milliseconds, meaning that this look-up and forwarding mechanism had virtually no impact on overall response times.

In summary, while it is true that the load-balancing approach has the propensity to add an overhead to response times, through the careful design of the load-balancing component architecture it has been demonstrated that the overheads introduced are very low, and are more than outweighed by the advantages of establishing the ability to meet SLAs under any workload through our load-balancing approach.

6.2.4 Discussion. The above experiments show that during the periods when upscaling and downscaling occurs, there may be (depending on configurations) a small delay, in the vicinity of 200–300 milliseconds, when a particular engine is unavailable to progress cases. While this may have a small negative impact on SLA throughput thresholds, it is usually the case that an SLA will not guarantee 100% conformance rate. Rather small, infrequent variations are absorbed by the contractual criteria of the SLA. So, from a service provider perspective, there is little to no risk of SLA violations occurring, nor therefore of any economic penalties being accrued by the provider or imposed by the client.

On the other hand, for the service provider, the renting of cloud resources can be expensive, and thus the periods of use for additional resources should be minimised. For these reasons, a service provider will enjoy the economic benefits delivered by our approach, by ensuring that cloud resources are used to a maximal capacity, and that whenever upscaling is required to ensure SLAs are not violated, the newly assigned resource is rented for the minimal period required. To this end, a detailed examination of the cost analysis and economic benefits of our approach will form an aspect of future work.

7 RELATED WORK

Addressing scalability of process execution in the cloud has emerged as one of the main topics in the field of BPM in the cloud [18]. So far, the majority of existing research efforts related to this topic *regard process execution as service composition*, where each task in a process is enacted by a (Web) service rather than a human resource. Such a service requires computational resources in the cloud and thereby such a process is also known as a *service-based process* [1]. In contrast, a business process often has an emphasis on the involvement of human resources and interactions with human users (e.g., managers, subject matter experts, operational staff), and the BPMS we study supports the execution of this kind of process. In addition, interoperability is often overlooked whilst scalability is being addressed in current studies in the field. To the best of our knowledge, existing proposals for developing cloud-based BPMS hardly consider a standard that specifies a generic system architecture of BPMS, such as WfMC's workflow reference model, as part of their underlying principles. Below, we review the related research efforts in more detail.

A main thread of research is carried out in the context of so-called *elastic processes*, a concept introduced by Dustdar et al. [5] to refer to the type of service-based processes that can be carried out using cloud resources to meet various requirements of cost and quality. Hoenisch et al. [11, 12] study how to execute elastic processes involving different control-flow constructs but with a limited level of complexity. The authors present an elastic BPMS that initially considers only processes of sequential tasks [12], and later the system was extended to handle processes with more basic structural constructs such as exclusive decisions, parallelism, and iteration [11]. To this end, the proposed design of BPMS for cloud in our study builds on WfMC's workflow reference model, and therefore it supports business processes that capture a comprehensive set of control-flow constructs as well as resource constructs (capturing interactions between systems and human resources) informed by well-established workflow patterns [24].

Other efforts along this thread of research, focus on how to improve and optimise (cloud) resource allocation for execution of elastic processes. In [6], the authors apply fuzzy theory to the design of a resource control mechanism which monitors process execution and determines cloud resource allocation. In [11], the authors propose an approach based on a cost optimisation model for scheduling the assignment of process instances to cloud resources. More recently, Rosinsky et al. [17] developed a linear optimisation model to compute a process enactment plan to achieve the minimal resource consumption for process execution during which process instances may need to migrate from one machine to another and migrations are capped to a limited number of times.

However, it is not clear whether these mechanisms support stateful process executions during resource allocation, and moreover, without an underpinning standard reference framework, whether or not these mechanisms can be applied in a more generic setting is yet unknown. To this end, the proposed design of BPMS for cloud in our study is independent of any specific system and platform, and our conceptual design specification can be used to serve as a reference for the development of specific cloud-based BPMS implementations that seek to provide full support for stateful process executions.

In addition to the above thread of research, the authors of [16] present a multi-tenant BPMS architecture, but utilising only a single-instance Apache ODE system (i.e., engine) in the cloud for multiple tenants to run their process instances on it, and the proposed architecture only achieves limited scalability. In [19], the authors propose an architecture specifically for hosting multiple YAWL engines in the cloud for multiple tenants, where only a basic load-balancing strategy is offered and scalability is not addressed. In [21], the authors present an approach that separates process execution and data management through the use of “self-guided artifacts.” It follows an artifact-centric paradigm for process execution and poses challenges for developing a new type of BPMS from the ground up without an existing standard framework.

The research reported in this article presents a substantial extension to our previous studies (*reference removed for blinded review*), where the system design proposed only focused on supporting large workloads by relying on a *fixed* number of active process engines. The proposed design of BPMS hereby is capable of both upscaling and downscaling to address dynamically varying demands of end users in the cloud, and has led to the implementation of a new prototype and subsequently the design and conduct of new experiments for proof-of-concept evaluation.

Finally, whilst not specifically geared towards cloud-based BPMS, the work on benchmarking BPMS is worth mentioning as it can serve as an important requirement to determine the optimal method for load balancing and costing, which may inform the future direction of our research in developing more precise load-balancing capabilities and costing strategies. More specifically, the study of Ferme et al. [7, 8] makes use of containers such as Docker to provide a test bed for producing repeatable and trusted benchmarking results. Their “BenchFlow” framework simulates users and interactions to enable meaningful comparisons between different systems. The same team of authors has also performed micro-benchmarking of some of the more common workflow patterns and discussed the effects that pattern complexity may have on performance metrics [20].

8 CONCLUSION

In this article, we have proposed both a conceptual design and a system realisation of scalable BPMS for deployment in a multi-tenanted cloud environment. Our conceptual design specification can be used to serve as a reference for the development of specific cloud-based BPMS implementations that seek to provide full stateful support for effective process executions. Our prototypical system implementation using an open source traditional BPMS platform demonstrated to achieve scalability and address dynamically varying requirements of end users.

The conceptual design proposed in this article helps to set out clear directions for future work. One is to develop more precise load-balancing capabilities, by considering more complex and accurate measures for runtime process model complexity metrics, and indeed for process engines’ busyness measures too. This aspect would include an exploration of the effects of case prioritisation on workloads and load balancing, and how upscaling would be triggered in such cases. Another is to develop a further capability level, one which interrogates process logs to determine actual experiential measures of past specification executions and uses that data as input into forecasting methods. A third is to apply machine learning techniques to gradually tune the load-balancing algorithms for maximal load smoothing across multiple workflow engines. A fourth is a thorough

exegesis of the economic benefits of the flexible upscaling and downscaling design proposed. Last but not least, further extensions to the proposed design to support security and privacy considerations set yet another important direction for future work.

REFERENCES

- [1] Mourad Amziani, Tarek Melliti, and Samir Tata. 2012. A generic framework for service-based business process elasticity in the cloud. In *Proceedings of the 10th International Conference on Business Process Management (BPM'12)*. 194–199.
- [2] Rajkumar Buyya, Satish Narayana Srirama, Giuliano Casale, Rodrigo N. Calheiros, Yogesh Simmhan, Blesson Varghese, Erol Gelenbe, Bahman Javadi, Luis Miguel Vaquero, Marco A. S. Netto, Adel Nadjaran Toosi, Maria Alejandra Rodriguez, Ignacio Martín Llorente, Sabrina De Capitani di Vimercati, Pierangela Samarati, Dejan S. Milojicic, Carlos A. Varela, Rami Bahsoon, Marcos Dias de Assunção, Omer Rana, Wanlei Zhou, Hai Jin, Wolfgang Gentzsch, Albert Y. Zomaya, and Haiying Shen. 2019. A manifesto for future generation cloud computing: Research directions for the next decade. *ACM Computing Survey* 51, 5 (2019), 105:1–105:38.
- [3] Frederick Chong and Gianpaolo Carraro. 2006. Architecture strategies for catching the long tail. *MSDN Library* (April 2006).
- [4] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. 2018. *Fundamentals of Business Process Management* (2nd ed.). Springer.
- [5] Shahram Dustdar, Yike Guo, Benjamin Satzger, and Hong Linh Truong. 2011. Principles of elastic processes. *IEEE Internet Computing* 15, 5 (2011), 66–71.
- [6] Seven Euting, Christian Janiesch, Robin Fischer, Stefan Tai, and Ingo Weber. 2014. Scalable business process execution in the cloud. In *2014 IEEE International Conference on Cloud Engineering*. 175–184.
- [7] Vincenzo Ferme, Ana Ivanchikj, and Cesare Pautasso. 2016. Estimating the cost for executing business processes in the cloud. In *International Conference on Business Process Management*. Springer, 72–88.
- [8] V. Ferme, A. Ivanchikj, C. Pautasso, M. Skouradaki, and F. Leymann. 2016. A container-centric methodology for benchmarking workflow management systems. In *Proceedings of the 6th International Conference on Cloud Computing and Service Science*. SciTePress, 74–84.
- [9] Terry Halpin. 2015. *Object-Role Modeling Fundamentals: A Practical Guide to Data Modeling with ORM*. Technics Publications, LLC.
- [10] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. 2004. Design science in information systems research. *MIS Quarterly* 28, 1 (2004), 75–105.
- [11] Philipp Hoenisch, Dieter Schuller, Stefan Schulte, Christoph Hochreiner, and Shahram Dustdar. 2016. Optimization of complex elastic processes. *IEEE Transactions on Services Computing* 9, 5 (2016), 700–713.
- [12] Philipp Hoenisch, Stefan Schulte, Shahram Dustdar, and Srikumar Venugopal. 2013. Self-adaptive resource allocation for elastic process execution. In *2013 IEEE 6th International Conference on Cloud Computing*. 220–227.
- [13] David Hollingsworth. 1995. *Workflow Management Coalition: The Workflow Reference Model*. Technical Report TC00-1003.
- [14] Kiranbir Kaur, Sandeep Sharma, and Karanjeet Singh Kahlon. 2017. Interoperability and portability approaches in inter-connected clouds: A review. *ACM Computing Surveys* 50, 4 (2017), 49:1–49:40.
- [15] OMG. December, 2017. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. Technical Report. <https://www.omg.org/spec/UML/2.5.1/PDF>.
- [16] Milinda Pathirage, Srinath Perera, Indika Kumara, and Sanjiva Weerawarana. 2011. A multi-tenant architecture for business process executions. In *2011 IEEE International Conference on Web Services*. IEEE, 121–128.
- [17] Guillaume Rosinosky, Samir Youcef, and F. Charoy. 2017. Efficient migration-aware algorithms for elastic BPaaS. In *Proceedings of the 15th International Conference on Business Process Management (BPM'17)*. 147–163.
- [18] Stefan Schulte, Christian Janiesch, Srikumar Venugopal, Ingo Weber, and Philipp Hoenisch. 2015. Elastic business process management: State of the art and open challenges for BPM in the cloud. *Future Generation Computer Systems* 46 (2015), 36–50.
- [19] D. M. M. Schunselaar, H. M. W. Verbeek, H. A. Reijers, and W. M. P. van der Aalst. 2014. YAWL in the cloud: Supporting process sharing and variability. In *International BPM Conference Workshops*. Springer, 367–379.
- [20] M. Skouradaki, V. Ferme, C. Pautasso, F. Leymann, and A. van Hoorn. 2016. Micro-benchmarking BPMN 2.0 workflow management systems with workflow patterns. In *28th International Conference on Advanced Information Systems Engineering*. Springer, 67–82.
- [21] Yutian Sun, Jianwen Su, and Jian Yang. 2016. Universal artifacts: A new approach to business process management (BPM) systems. *ACM Transactions on Management Information Systems* 7, 1 (2016), 3:1–3:26.
- [22] Arthur H. M. ter Hofstede, Wil M. P. van der Aalst, Michael Adams, and Nick Russell (Eds.). 2010. *Modern Business Process Automation—YAWL and Its Support Environment*. Springer.

- [23] W. M. P. van der Aalst and A. H. M. ter Hofstede. 2005. YAWL: Yet another workflow language. *Information Systems* 30, 4 (June 2005), 245–275.
- [24] Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, Bartek Kiepuszewski, and Alistair P. Barros. 2003. Workflow patterns. *Distributed and Parallel Databases* 14, 1 (2003), 5–51.
- [25] John Venable. 2006. A framework for design science research activities. In *2006 Information Resource Management Association International Conference*. Idea Group Publishing, 184–187.
- [26] Mathias Weske. 2019. *Business Process Management—Concepts, Languages, Architectures*. (3rd ed.) Springer.

Received October 2019; revised March 2021; accepted April 2021